

Binary Numbers

ACOL215: Digital Logic And System Design
Chapter 1

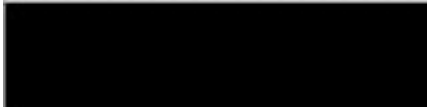
Abhilash Jindal

Digital systems

- Represent and manipulate *discrete elements* (set) of information. Example: *digits (hence digital systems)*, letters of alphabet, playing cards, chessboards squares
- Discrete elements are represented by electrical signals (voltages/current)
- Signals are controlled and manipulated by transistors

Discrete quantities of information

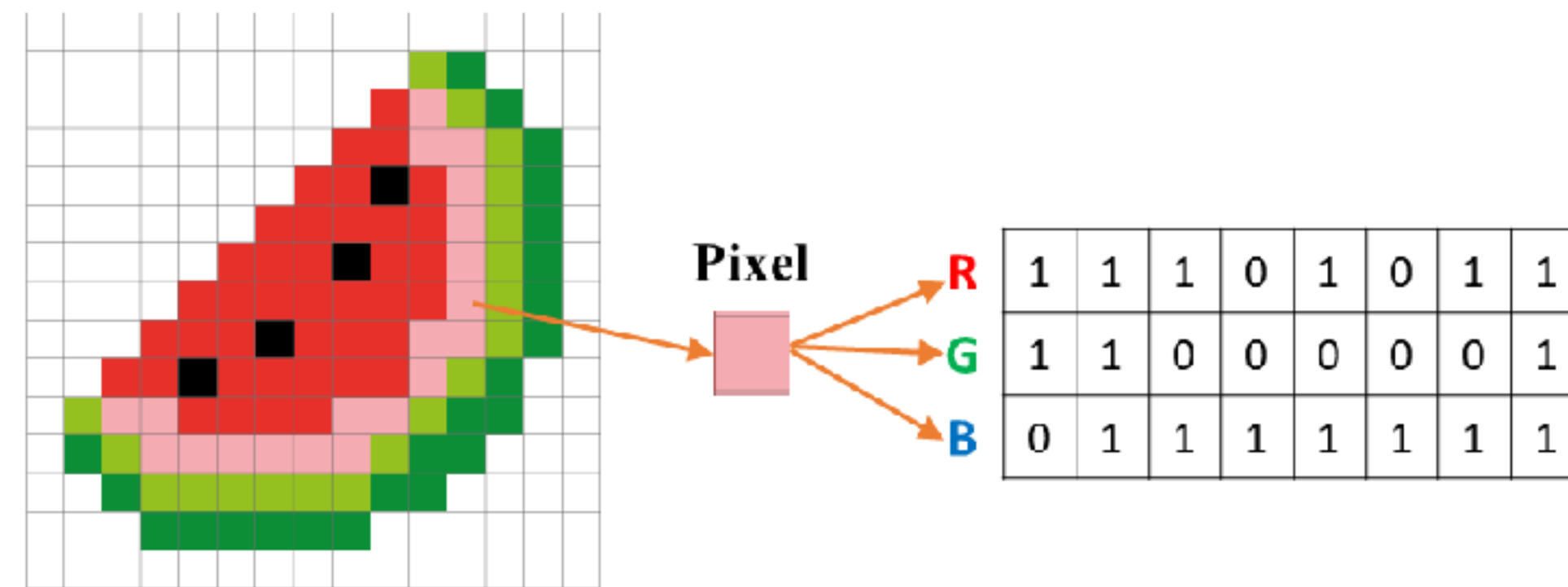
- Inherently discrete processes
 - Payroll management: Employee names, PAN Card number, monthly salaries, income tax deduction
- Continuous processes
 - Example: Digital camera.
 - Colors are discretised into numbers
 - Images are discretised into pixels, where each pixel has a discretised color

	Red	Green	Blue
	0	0	0
	255	255	255
	255	0	0
	0	255	0
	0	0	255
	255	128	0
	255	255	0
	128	128	128



Present-day digital systems

- Use just two discrete values: on<>off
- Binary digit (bit) has two numerical values: 0 or 1
- Discrete elements of information are represented with group of bits



Decimal numbers

How to represent numbers with bits?

- Consider 7,392.54
- $= 7000 + 300 + 90 + 2 + 0.5 + 0.04$
- $= 7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0 + 5 \times 10^{-1} + 4 \times 10^{-2}$
- In general, a number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots =$
 $\cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- For 7,392: $a_3 = 7; a_2 = 3; a_1 = 9; a_0 = 2$. Rest are zero.

Radix

- A decimal number $\cdots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \cdots = \cdots + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots$
- Decimal numbers have **10** symbols: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.
- Each coefficient is multiplied with a power of **10**.
- Number is in base 10 or radix 10
- Written as $(7,392.54)_{10}$

Base-r number system

- Base-r numbers have **r** symbols: $\{0, \dots, r - 1\}$. $a_j \in \{0, \dots, r - 1\}$
- A base-r number $\dots a_3 a_2 a_1 a_0 . a_{-1} a_{-2} \dots =$
 $\dots + a_3 \times r^3 + a_2 \times r^2 + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \dots$
- Each coefficient is multiplied with a power of **r**.

Examples

- Base-5 numbers have 5 symbols $\{0, \dots, 4\}$. $(4021.2)_5 = ?$
 - $4 * 5^3 + 0 * 5^2 + 2 * 5^1 + 1 * 5^0 + 2 * 5^{-1} = (511.4)_{10}$
- Octal numbers (base-8) have 8 symbols $\{0, \dots, 7\}$. $(127.4)_8$
 - $1 * 8^2 + 2 * 8^1 + 7 * 8^0 + 4 * 8^{-1} = (87.5)_{10}$

Binary numbers

- Binary numbers (base-2) have 2 symbols $\{0,1\}$. $(11010.11)_2 = ?$
 - $1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 + 1 * 2^{-1} + 1 * 2^{-2} = (26.75)_{10}$
- $(10101)_2 = ?$
 - $(21)_{10}$
- $(110101)_2 = ?$
 - $(53)_{10}$

Hexadecimal numbers

- Hexadecimal numbers (base-16) have 16 symbols $\{0, \dots, 9, A, B, C, D, E, F\}$.
 $(B65F)_{16} = ?$
 - $11 * 16^3 + 6 * 16^2 + 5 * 16^1 + 15 * 16^0 = (46,687)_{10}$

Arithmetic operations on base-r numbers

Addition

- $101101 + 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- When sum is $2_{10} (=10_2)$, write 0 and carry 1.
When sum is $3_{10} (=11_2)$, write 1 and carry 1
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(1010100)_2 = (84)_{10}$

Arithmetic operations on base-r numbers

Subtraction

- $101101 - 100111 = ?$
- One must be careful to only allow r-allowable digits: $\{0, \dots, r - 1\}$
- In decimal, say 96-47.
 - When we encounter “6-7”, put (10-1) in output and reduce previous digit by 1, i.e., 9 to 8.
- In binary, 0-0 is 0. 1-1 is 0. 1-0 is 1.
When encountering 0-1: write (2-1) in output and reduce previous bit by 1.
- $(101101)_2 = (45)_{10}$, $(100111)_2 = (39)_{10}$, $(000110)_2 = (6)_{10}$

Arithmetic operations on base-r numbers

Multiplication

- $1011 * 101 = ?$
- One must be careful to only allow r -allowable digits: $\{0, \dots, r - 1\}$
- When multiplier digit is 1, copy multiplicand in the partial product. Shift multiplicand left for every bit. Compute addition of all partial products
- $(1011)_2 = (11)_{10}$, $(101)_2 = (5)_{10}$, $(110111)_2 = (55)_{10}$

N-bit binary numbers

- 3-digit decimal numbers can represent 10^3 numbers: $\{000, 001, \dots, 999\}$
- 3-bit binary numbers can represent 2^3 numbers: $\{000, 001, \dots, 111\}$ i.e., $\{0, \dots, 7\}$
- A byte has 8-bits. It can represent 2^8 numbers: $\{0, \dots, 255\}$. Or $\{0, \dots, FF\}$ in hexadecimal.
 - Enough to represent a keyboard character

Kilo-Mega-Giga-Tera

Larger groups of bits

- Kilo (K): 2^{10} , Mega (M): 2^{20} , Giga (G): 2^{30} , Tera (T): 2^{40}
- 1MB of file contains 2^{20} bytes = ~1 Million bytes.
How many *bits* can 1TB of disk store?

Table 1.1
Powers of Two

<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>	<i>n</i>	<i>2ⁿ</i>
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

Characters

ASCII character code

- 01/char.c

Table 1.7
American Standard Code for Information Interchange (ASCII)

<i>b₄b₃b₂b₁</i>	<i>b₇b₆b₅</i>							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	‘	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L		l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

Error-Detecting Code

- ASCII is a 7-bit code. 8th *parity bit* is sometimes added to check for transmission/storage retrieval errors.
- Odd parity: Parity bit is **0** if number of 1s are **odd**, 1 otherwise
- Even parity: Parity bit is **0** if number of 1s are **even**, 1 otherwise

		Even parity	Odd parity
ASCII A	1000001	01000001	11000001
ASCII T	1010100	11010100	01010100

- What is the even parity of 0101100?

Converting decimal integers to binary

- How to find coefficients $\cdots a_2 a_1 a_0 . a_{-1} a_{-2} \cdots = \cdots + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0 + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \cdots$
- Continue dividing the number and all successive quotients by 2 until the quotient reaches 0
- $(41)_{10} = ?$
- $(101001)_2$

	Quotient	Remainder
41/2	20	1
20/2	10	0
10/2	5	0
5/2	2	1
2/2	1	0
1/2	0	1



Converting decimal integers to octal

- $(153)_{10} = ?$
- $(231)_8$
- $(135)_{10} = (??)_2$
- $(135)_{10} = (??)_8$

	Quotient	Remainder
153/8	19	1
19/8	2	3
2/8	0	2

Converting decimal *fraction* to binary

- Multiply the number and each successive fraction with 2 at each step, until the fraction reaches 0 or until enough significant coefficients are obtained
- $(0.6875)_{10} = ?$
- $(0.1011)_2$

	Integer	Fraction
$0.6875 * 2$	1	0.3750
$0.3750 * 2$	0	0.75
$0.750 * 2$	1	0.5
$0.500 * 2$	1	0



Converting decimal *fraction* to octal

- $(0.513)_{10} = ?$
- $(0.406517..)_{8}$
- Overall: separate integer and fraction parts. Convert them to base-r numbers separately, and recombine them.
- $(153.513)_{10} = (231.406517)_{8}$

	Integer	Fraction
$0.513 * 8$	4	0.104
$0.104 * 8$	0	0.832
$0.832 * 8$	6	0.656
$0.656 * 8$	5	0.248
$0.248 * 8$	1	0.984
$0.984 * 8$	7	0.872
...	...	...

Representing long string of bits

- While computers work with binary numbers, all human interfaces specify long bits using octal/hexadecimal/or even base-256.
- Instead of writing a color of 111111110000000011111111, it is more convenient to write FF00FF

	Red	Green	Blue	Hexadecimal code
	0	0	0	#000000
	255	255	255	#FFFFFF
	255	0	0	#FF0000
	0	255	0	#00FF00
	0	0	255	#0000FF
	255	128	0	#FF8000
	255	255	0	#FFFF00
	128	128	128	#808080

Representing long string of bits

Octal and Hexadecimal numbers

- Shorter patterns of characters are easier to identify than long patterns of 1's and 0's
- Since $2^3 = 8$, each octal number represents 3 bits
- Since $2^4 = 16$, each hexadecimal number represents 4 bits

Decimal	Binary	Octal	Hexadecimal
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Converting binary to octal/hexadecimal

- Partition into groups of 3 bits. Assign the corresponding octal digit to each group
- $(10110001101011.111100000110)_2$ to octal?

10	110	001	101	011	.	111	100	000	110
2	6	1	5	3	.	7	4	0	6

- $(10110001101011.11110010)_2$ to hexadecimal?

10	1100	0110	1011	.	1111	0010
2	C	6	B	.	F	2

Converting octal/hexadecimal to binary

- Reverse the procedure: replace each octal digit with its 3-bit binary equivalent. Each hexadecimal digit with its 4-bit binary equivalent
- $(673.124)_8$ to binary?

6	7	3	.	1	2	4
110	111	011	.	001	010	100

- $(306.D)_{16}$ to binary?

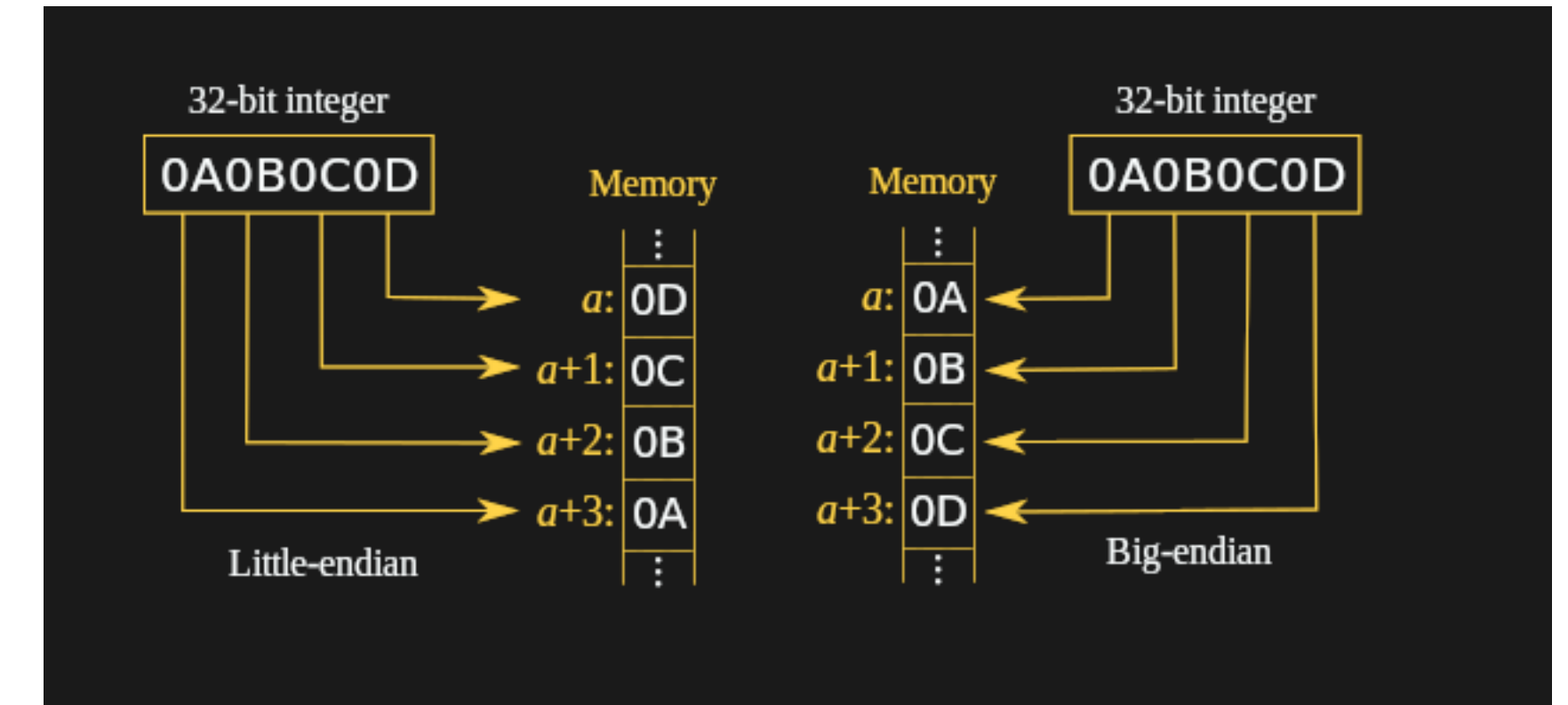
3	0	6	.	D
0011	0000	0110	.	1101

Representing long string of bits (2)

- An IPv4 address is a base-256 number: $(10.237.23.38)_{256}$
- Convert to binary/hexadecimal?
- How many unique IPv4 addresses can be there?

Whole numbers

- 01/unsigned.c
- unsigned int is 4 bytes. It can store $\{0, \dots, 2^{32} - 1\}$
- unsigned long is 8 bytes. It can store $\{0, \dots, 2^{64} - 1\}$
- On unsigned int, are + and * commutative and associative?
- Does + have an identity element? Inverses? Is $x + 1$ always greater than x ?
- Does * have an identity element? Inverses?



Integers

- 01/ints.c
 - How to store negative numbers? -17 seems to invert all digits of 17?
 - INT_MAX has most-significant bit 0. INT_MIN has it as 1
- int is 4 bytes. It can store $\{-2^{31}, \dots, 2^{31} - 1\}$
- short is 2 bytes. long is 8 bytes

How to store negative numbers?

Signed binary numbers

- To represent negative numbers, we need a notation for negative values: need to use bits to also represent the sign (+/-)
- *Signed-magnitude system*: Leftmost bit is 0 for positive, and 1 for negative numbers.
 - +9 is 01001. -9 is 11001.

Problems with signed magnitude system

- 0 is 00000. What is 10000?
- Addition follows the rules of ordinary numbers:
 - If signs are same, $(-37) + (-25) = -62$. Do an addition. Output carries the sign of inputs.
 - If signs are different, find which number is larger, then do subtraction.
 $(+25) + (-37) = -(37 - 25) = -12$
- Addition is awkward to do in hardware: first do comparisons of signs and magnitudes, perform either addition or subtraction, apply sign to the output

How to store negative numbers?

Complements of binary numbers

- 1's complement: change all 1's to 0's and 0's to 1's.
- For an n-digit binary number N, it is equivalent to $(2^n - 1) - N$. $2^n - 1$ is a string of (n-1) 1's
- 2's complement: Add 1 to 1's complement. For an n-digit number N, it is equivalent to $2^n - N$.
- 2's complement of 1101100
 - 0010100

2's complement of a 2's complement

- 2's complement of 0010100
 - 1001001
 - 2's complement of 1001001 is 0010100
- Complement of the complement restores the original number. $2^n - (2^n - N)$

Subtraction with complements

- Using complements, subtraction can be achieved via an addition circuit
- Subtraction of two n-digit unsigned binary numbers M-N can be done by:
- Add M to 2's complement of N. $M + (2^n - N)$
 - If $M > N$, the sum will produce an end carry 2^n . Discard it.
- $X = 1010100$. $Y = 1000011$. $X - Y$

$$\begin{array}{rcl} X & = & 1010100 \\ \text{2's complement of } Y & = & +\underline{0111101} \\ \text{Sum} & = & 10010001 \\ \text{Discard end carry } 2^7 & = & -10000000 \\ \text{Answer: } X - Y & = & 0010001 \end{array}$$

Subtraction with complements (2)

- Add M to 2's complement of N. $M + (2^n - N)$
 - If $M < N$, the sum will not produce an end carry 2^n and is instead equal to $2^n - (N - M)$. Take a 2's complement and add a negative sign in front.

- $X = 1010100$. $Y = 1000011$. $Y - X$

(b)

$$\begin{array}{r} Y = 1000011 \\ 2\text{'s complement of } X = +0101100 \\ \hline \text{Sum} = 1101111 \end{array}$$

There is no end carry. Therefore, the answer is $Y - X = -(2\text{'s complement of } 1101111) = -0010001$.

Exercise: $X = 1101010$, $Y = 0101011$
Find $X - Y$ and $Y - X$

Signed-2's-complement system

- Negative number is represented by its two's complement.
- +6 is 0110. -6 is 1010. Left-most digit still represents the sign.
- 0000 is 0. 1000 is -8

Table 1.3
Signed Binary Numbers

Decimal	Signed-2's Complement
+7	0111
+6	0110
+5	0101
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
−0	—
−1	1111
−2	1110
−3	1101
−4	1100
−5	1011
−6	1010
−7	1001
−8	1000

Signed-2's-complement system simplifies arithmetic

- Addition of two signed-2's-complement numbers is obtained simply from their addition, including the sign bits.

$+ 6$	
$+13$	
$\hline +19$	
$+ 6$	
-13	
$\hline - 7$	

$- 6$	
$+13$	
$\hline +7$	
$- 6$	
-13	
$\hline -19$	

Negative result is automatically in
its 2's-complement form

A carry out of the sign-bit
position is discarded

Addition hardware is sufficient to perform subtraction!

- To compute $A - B$, just compute $A + (-B)$ where $-B$ is just 2's complement of B .
- Say we have to compute $(-6) - (-13)$, i.e., $(11111010 - 11110011)$
- Compute $(11111010 + 00001101) = 1\ 00000111$. Discarding the end carry, we see that it is $+7$.
- Understand rest of `01/ints.c`

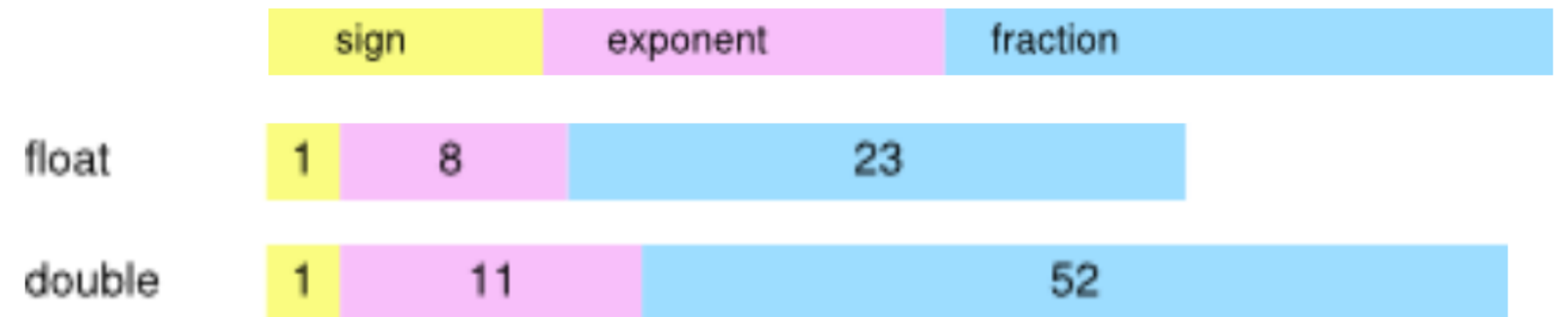
Real numbers

IEEE 754 standard

- In any non-trivial range, there are *infinite* real numbers
- How to squeeze infinitely many numbers into a finite number of bits (say 32)?
- Some programs work with very large numbers,
e.g., number of atoms in a mole: 6.022×10^{23} .
- Others with very small numbers,
e.g., charge of an electron 1.602×10^{-19}

Floating point representation

- Float: 32 bits, double: 64 bits
- Sign bit is 1 for negative numbers, 0 otherwise
- Exponent encodes the exponent in base 2
- Fraction encodes the significand without the most significant non-zero bit



Floating point representation (2)

- $-192 = -1 * 128 * 1.5 = -1 * 2^7 * (1.1)_2$
- For float, exponent is biased by 127 (0111111), i.e., to represent 7 store 134
- For double, exponent is biased by 1023, i.e., to represent 7 store 1030



-192

Examples representations

01/floats.c

- 0.0: all zeros. -0.0: all zeros but with sign bit 1
- 1.0: $2^0 * 1.0$. Exponent is 127. Mantissa ignores leading 1, hence all 0s.
- 0.5: $2^{-1} * 1.0$. Exponent is 126.
- Special numbers:
 - 1/0: Infinity. All 1s in exponent. All zeros in fraction
 - -Infinity. Just turn on sign bit
 - Sqrt of negative number: NaN. All 1s in exponent. MSB is 1 in fraction.

Examples representations (2)

- Largest number: $(1.11111111111111111111111111111111)_2 * 2^{127}$. All 1s in fraction. Leave 1 in exponent for infinity.
- Smallest positive number: $1.0 * 2^{-126}$. All 0s in fraction. Leave 0 in exponent for 0.

Rounding errors

01/floats.c

- Fitting infinitely many numbers in a finite range leads to rounding approximations:
 - $(0.1)_{10}$
 - $2^{-3} * (1.100110011001...)_{2}$
 - Approximated to $(0.10000000149011611938477)_{10}$
 - $0.1 + 0.2 \neq 0.3$

	Integer	Fraction
$0.1 * 2$	0	0.2
$0.2 * 2$	0	0.4
$0.4 * 2$	0	0.8
$0.8 * 2$	1	0.6
$0.6 * 2$	1	0.2
$0.2 * 2$	0	0.4
$0.4 * 2$	0	0.8
$0.8 * 2$	1	0.6

Addition is not associative due to rounding approximations!

- $A = 2^1 * 1.0001; B = 2^0 * 1.0001; C = 2^3 * 1.0001$
 - $A + B = 2^1 * 1.10011; rn(A + B) = 2^1 * 1.1010 = 2^3 * 0.011010;$
 $rn(A + B) + C = 2^3 * 1.01111; rn(rn(A + B) + C) = 2^3 * 1.1000$
 - $B + C = 2^3 * 1.0011001; rn(B + C) = 2^3 * 1.0011$
 $A + rn(B + C) = 2^3 * 1.01110001; rn(A + rn(B + C)) = 2^3 * 1.0111$
- rn rounds the number to nearest floating point. $rn(A+B)$ got rounded up. $rn(B+C)$ got rounded down.
- Order of operations can affect accuracy of results!

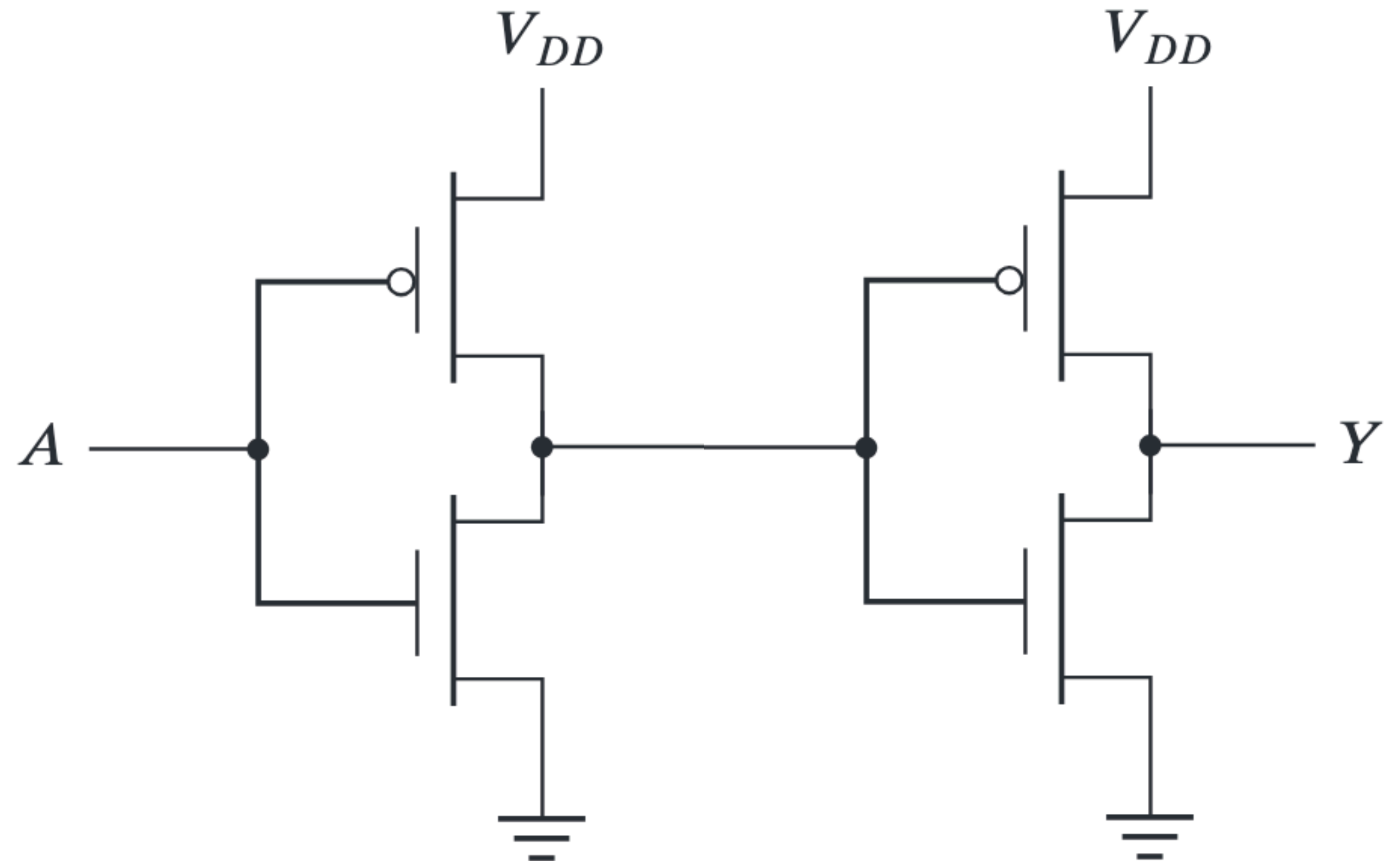
Summary

- Present-day digital systems use just two discrete values: on<>off. Binary digit (bit) has two numerical values: 0 or 1. Discrete elements of information are represented with group of bits
- Learned how to represent
 - characters: ASCII table
 - whole numbers: Unsigned ints
 - signed numbers: 2's complements, ints
 - real numbers: exponent-fraction form in floats
- Representations can break our assumptions in math. $x+1$ can be less than x .
 $0.1+0.2$ is not 0.3

What's next?

Binary logic

- $Y = ?$
- Reason about circuits at a higher-level instead of thinking in terms of voltages and transistors
- Binary logic
 - binary variables (A and Y) that take on two discrete values
 - Operations on binary variables



NOT/AND/OR

- NOT. $z = x'$ or $z = \bar{x}$. Output is 1 iff input is 0
- AND. $z = x \cdot y$. Output is 1 iff both inputs are 1.
- OR. $z = x + y$. Output is 1 iff any input is 1. In binary logic, $1+1=1$. In binary arithmetic, $1+1=10$

Table 1.8
Truth Tables of Logical Operations

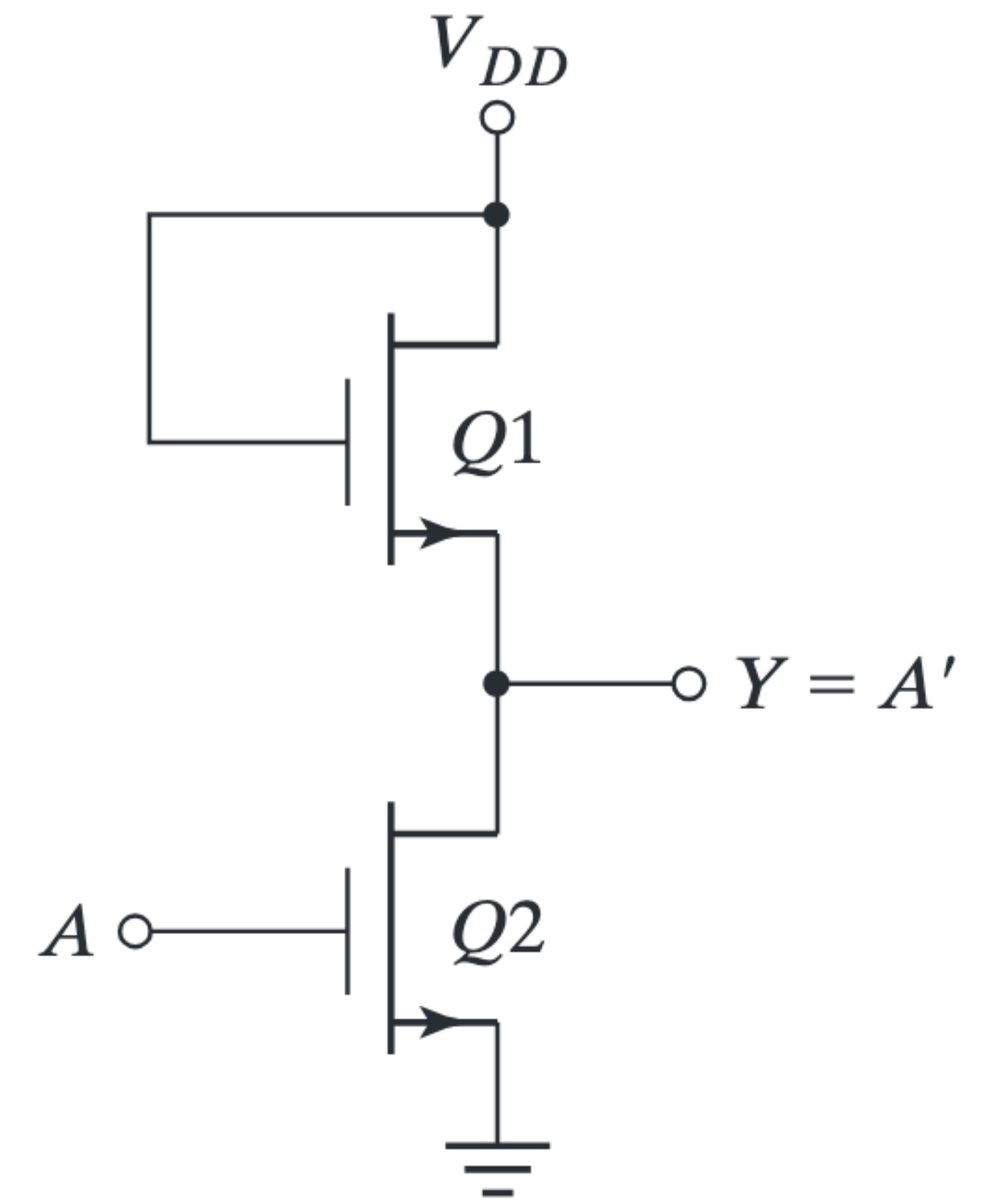
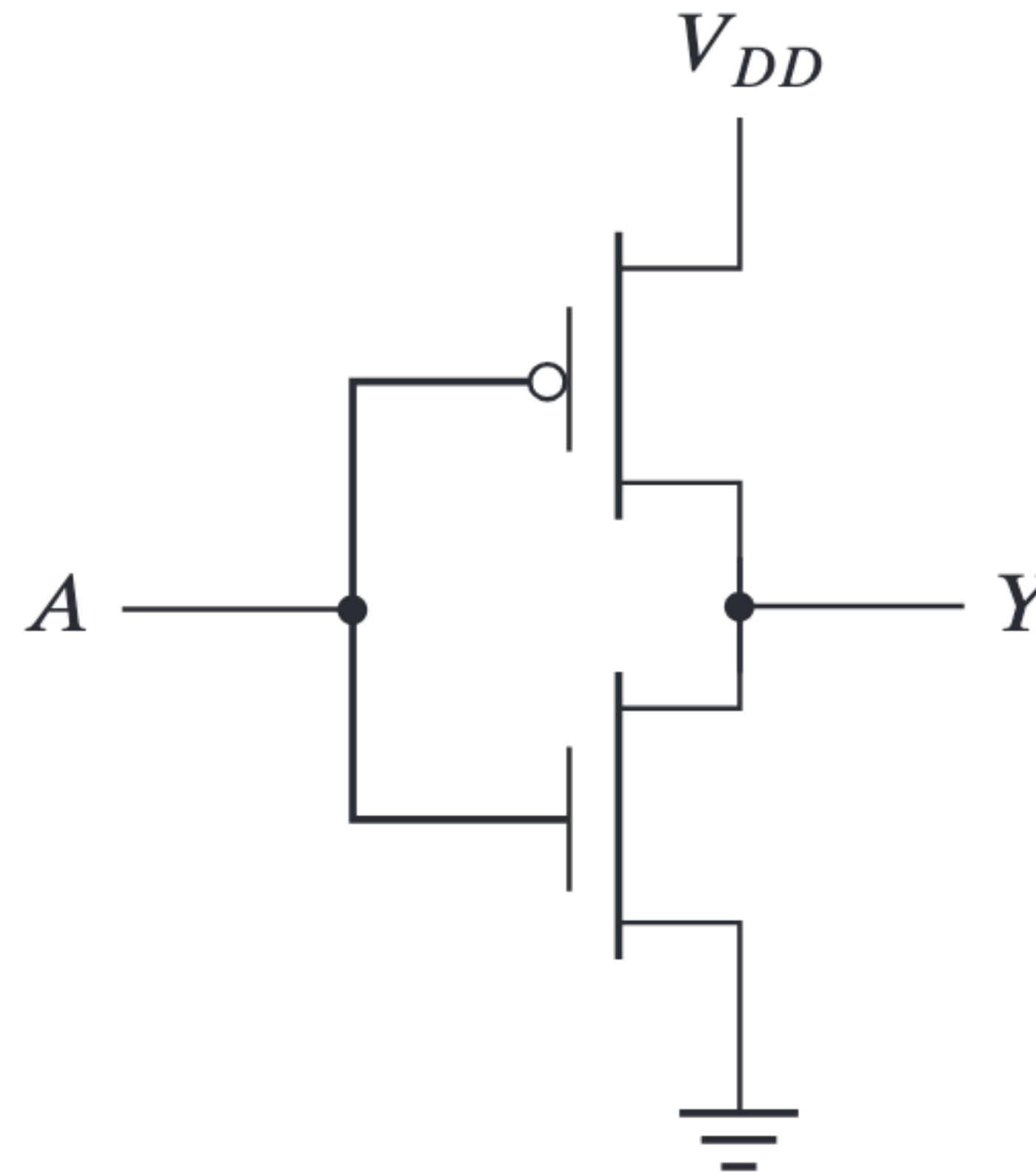
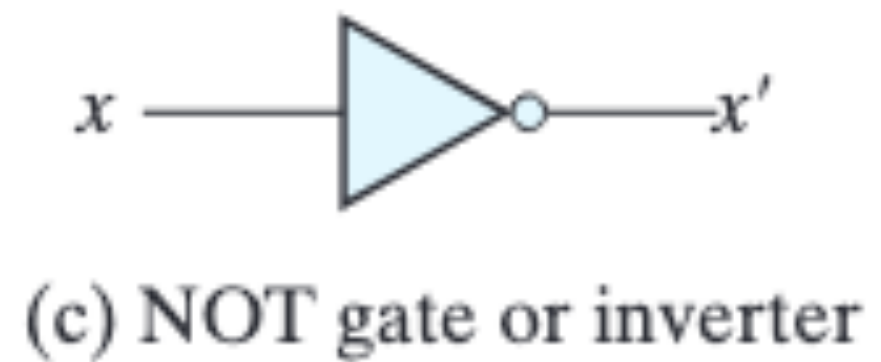
AND			OR			NOT	
x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Logic gates

Inverter

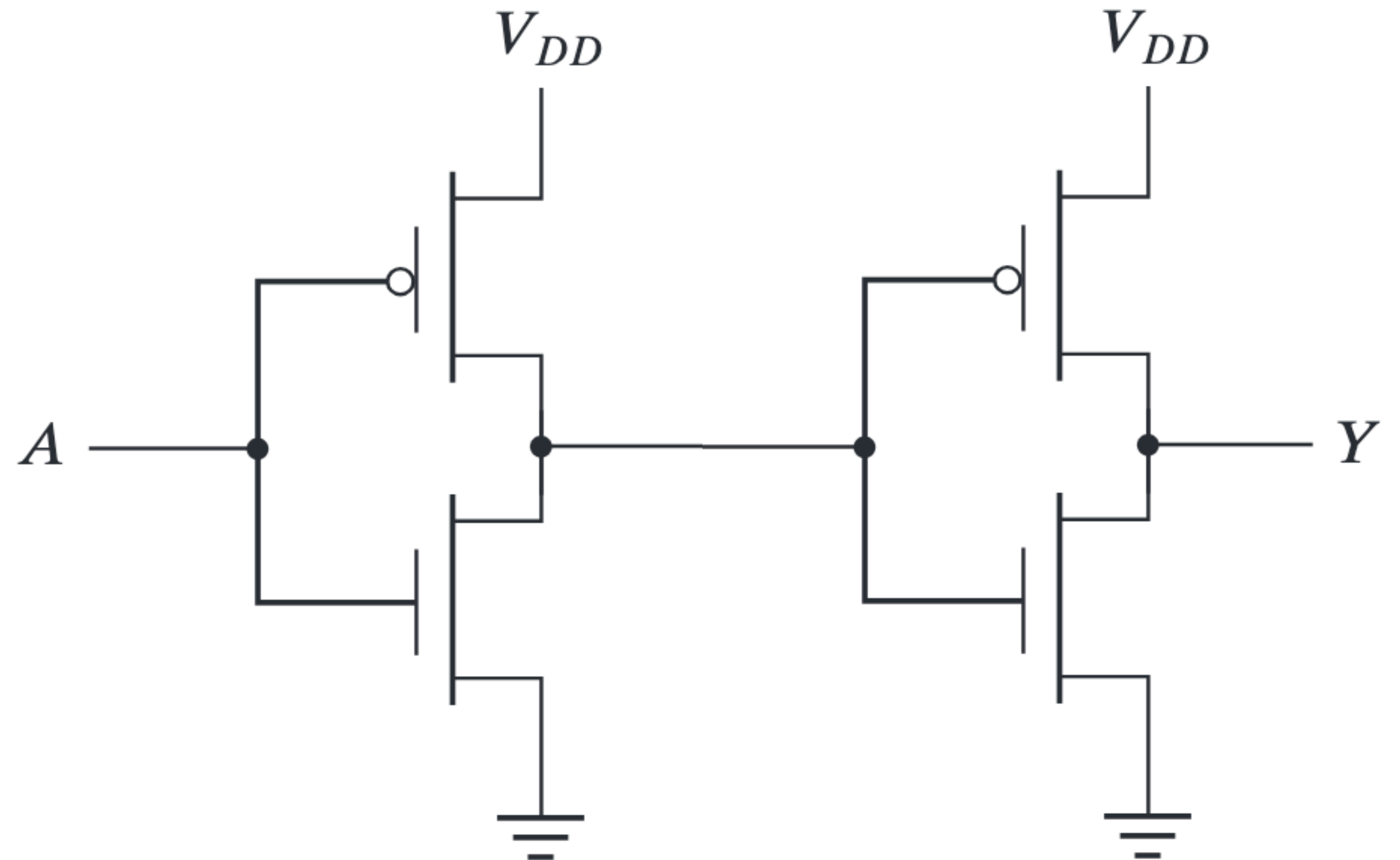
- Abstract away from the circuit-level implementation
- $Y = A'$ or $\bar{A}$

NOT	
x	x'
0	1
1	0



Binary logic

- Reason about circuits at a higher-level instead of thinking in terms of voltages and transistors
- $Y = (A')' = A$



Circuit simulations

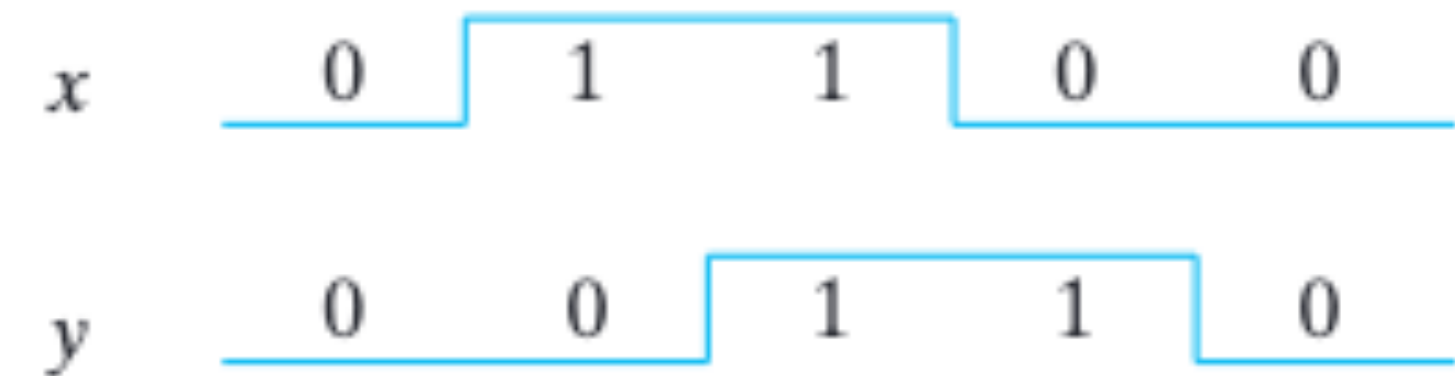
- Timing diagrams: simulate circuits to understand its response to changing inputs



(a) Two-input AND gate



(b) Two-input OR gate



AND: $x \cdot y$

OR: $x + y$

NOT: x'